



LABORATORIO: FILE PATH TRAVERSAL - VALIDATION OF FILE EXTENSION WITH NULL BYTE BYPASS

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ

185583

ING. EN TECNOLOGÍAS DE LA INFORMACIÓN



Índice

Especificaciones del laboratorio.....	2
Marco teórico.....	2
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento	4
1. Acceso al laboratorio mediante Burp Suite	4
2. Identificación de las peticiones de imágenes.....	4
3. Envío de la petición a Repeater	5
4. Modificación del parámetro vulnerable	6
5. Análisis de la respuesta	6
6. Confirmación de resolución	7
Explicación del payload.....	8
Mitigación recomendada	8
Referencias	9

Especificaciones del laboratorio

Nombre del laboratorio: File path traversal, validation of file extension with null byte bypass.

Nivel: PRACTITIONER

Tipo de explotación: File Path Traversal (Directory Traversal) mediante bypass de validación del inicio de la ruta.

Marco teórico

El **File Path Traversal**, también conocido como **Directory Traversal**, es una vulnerabilidad que ocurre cuando una aplicación utiliza datos proporcionados por el usuario para construir rutas dentro del sistema de archivos sin comprobar adecuadamente que el recurso solicitado permanezca dentro del directorio autorizado.

Una técnica convencional para explotar esta vulnerabilidad consiste en utilizar secuencias como `../`, donde `..` representa el directorio superior y `/` funciona como separador. La repetición de estas secuencias permite retroceder progresivamente dentro de la estructura de directorios e intentar acceder a archivos que no deberían encontrarse disponibles para el usuario.

Para reducir este riesgo, algunas aplicaciones implementan mecanismos de validación adicionales. Una estrategia consiste en comprobar que el nombre del archivo solicitado termine con una extensión permitida, por ejemplo `.png`. De esta manera, una aplicación destinada a recuperar imágenes podría rechazar una entrada como: `../../../../etc/passwd` debido a que no termina con la extensión requerida.

Sin embargo, determinadas implementaciones pueden presentar diferencias entre la forma en que la aplicación valida una cadena y la forma en que posteriormente es interpretada por componentes que procesan la ruta. Históricamente, uno de estos casos involucra el null byte o carácter nulo, representado por el valor `0x00` y que puede aparecer codificado en una URL como `%00`.

En determinados contextos, el carácter nulo puede ser interpretado como el final de una cadena. Esto puede producir una discrepancia: la validación de la aplicación puede observar una cadena completa que aparentemente termina con una extensión permitida, mientras que otro componente puede considerar que el nombre del archivo termina en la posición del null byte.

Por ejemplo: `../../../../etc/passwd%00.png`. La cadena contiene `.png`, pero si el carácter nulo es interpretado como terminador, la parte posterior puede dejar de formar parte del nombre utilizado para acceder al sistema de archivos. De esta manera, el recurso procesado efectivamente puede corresponder a `/etc/passwd`.

Desde el modelo de seguridad **CIA (Confidentiality, Integrity and Availability)**, esta vulnerabilidad afecta principalmente a la **confidencialidad**, ya que puede permitir el acceso a archivos del sistema, configuraciones, código fuente, registros o credenciales que deberían permanecer fuera del alcance del usuario.

La prevención requiere evitar diferencias entre la validación realizada por la aplicación y la interpretación posterior de la ruta. Las entradas deben validarse de forma estricta y la aplicación debe comprobar la ubicación final del recurso antes de acceder al sistema de archivos.

Objetivo del laboratorio

Identificar y explotar de manera controlada una vulnerabilidad de **File Path Traversal** presente en el mecanismo utilizado por una aplicación web para cargar imágenes de productos.

Mediante **Burp Suite**, se busca analizar y modificar una petición HTTP que contiene el parámetro **filename**. La aplicación comprueba que el nombre del archivo solicitado termine con una extensión de imagen permitida; sin embargo, esta validación puede ser evadida utilizando un **null byte** antes de la extensión esperada.

El laboratorio se considera resuelto al modificar el parámetro mediante el payload: **../../../../etc/passwd%00.png** y conseguir que el servidor devuelva el contenido del archivo **/etc/passwd**, demostrando que la validación de extensión implementada no impide adecuadamente el acceso a archivos fuera del directorio autorizado.

Explicación técnica de la vulnerabilidad

La aplicación carga las imágenes de los productos mediante peticiones HTTP dirigidas al endpoint **/image**, utilizando el parámetro **filename** para indicar qué archivo debe recuperar el servidor.

Una petición legítima presenta una estructura similar a: **GET /image?filename=10.jpg**.

En este laboratorio, la aplicación incorpora una validación destinada a comprobar que el archivo solicitado tenga una extensión correspondiente a una imagen. Por esta razón, proporcionar directamente una ruta como: **../../../../etc/passwd** no resulta suficiente, debido a que el valor no termina con la extensión esperada.

La vulnerabilidad aparece porque es posible introducir un **null byte** entre el nombre del archivo que se desea recuperar y una extensión aparentemente válida.

El carácter nulo puede representarse mediante: **%00**. Por esta razón, el parámetro puede construirse de la siguiente manera: **../../../../etc/passwd%00.png**.

Desde el punto de vista de la validación, la entrada contiene la extensión **.png**. Sin embargo, en el entorno vulnerable del laboratorio, el carácter nulo permite que el procesamiento del nombre del archivo termine antes de dicha extensión.

Conceptualmente, la aplicación recibe: **../../../../etc/passwd%00.png**, pero el recurso que termina siendo recuperado corresponde a: **../../../../etc/passwd**.

Las secuencias `../../../../` permiten abandonar el directorio destinado al almacenamiento de imágenes y alcanzar otras ubicaciones del sistema. Posteriormente, `etc/passwd` identifica el archivo que se desea recuperar.

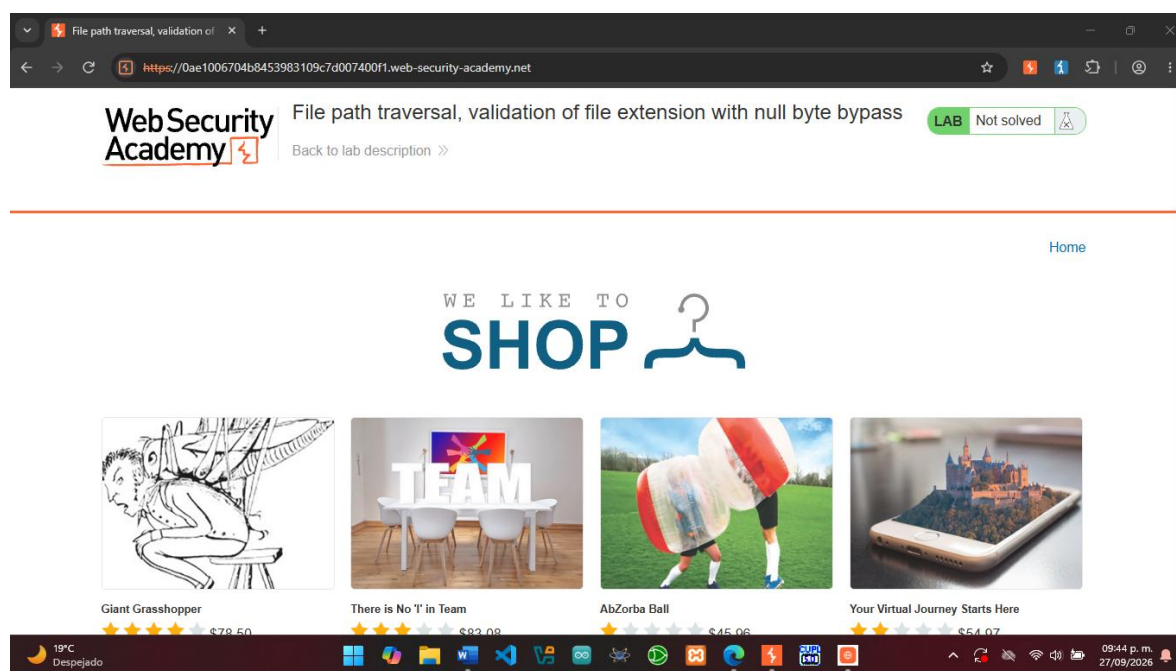
Por lo tanto, la vulnerabilidad surge porque existe una diferencia entre la validación de la extensión y la interpretación final del nombre del archivo, permitiendo conservar una extensión aparentemente válida sin que esta forme parte efectiva del recurso recuperado.

Procedimiento

1. Acceso al laboratorio mediante Burp Suite

Se abrió **Burp Suite Community Edition** y, desde la pestaña **Proxy**, se inició el navegador Chromium integrado mediante la opción **Open Browser**. Este navegador se encuentra configurado para dirigir su tráfico a través del proxy de Burp Suite, permitiendo observar las peticiones y respuestas HTTP generadas durante la navegación.

Posteriormente, desde dicho navegador se accedió a la instancia asignada del laboratorio **File path traversal, validation of file extension with null byte bypass** de PortSwigger Web Security Academy.



2. Identificación de las peticiones de imágenes

Desde **Proxy > HTTP history** se analizaron las solicitudes generadas por la aplicación. Se observó que la carga de las imágenes de los productos producía múltiples peticiones **GET** dirigidas al endpoint `/image`, cada una de ellas utilizando el parámetro `filename` para indicar el archivo que debía ser recuperado. Por ejemplo: `GET /image?filename=10.jpg`.

Este comportamiento permitió identificar **filename** como un parámetro susceptible de prueba, debido a que su valor determina el archivo solicitado al servidor.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type
1	https://0ae1006704b845398...	GET	/			200	10591	HTML
6	https://0ae1006704b845398...	GET	/image?filename=10.jpg	✓		200	146550	JPEG
8	https://0ae1006704b845398...	GET	/image?filename=33.jpg	✓		200	198033	JPEG
10	https://0ae1006704b845398...	GET	/image?filename=67.jpg	✓		200	196990	JPEG
12	https://0ae1006704b845398...	GET	/image?filename=58.jpg	✓		200	163688	JPEG
16	https://0ae1006704b845398...	GET	/academyLabHeader			101	147	
17	https://0ae1006704b845398...	GET	/image?filename=6.jpg	✓		200	118082	JPEG
18	https://0ae1006704b845398...	GET	/image?filename=50.jpg	✓		200	271256	JPEG
19	https://0ae1006704b845398...	GET	/image?filename=66.jpg	✓		200	117924	JPEG
20	https://0ae1006704b845398...	GET	/image?filename=15.jpg	✓		200	313445	JPEG
21	https://0ae1006704b845398...	GET	/image?filename=59.jpg	✓		200	182027	JPEG

3. Envío de la petición a Repeater

Se seleccionó una de las peticiones dirigidas a **/image** y se utilizó la opción **Send to Repeater**, obtenida del menú desplegado al dar clic derecho sobre la petición.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type	Extension
1	https://0ae1006704b845398...	GET	/			200	10591	HTML	
6	https://0ae1006704b845398...	GET	/image?filename=10.jpg						
8	https://0ae1006704b845398...	GET	/image?filename=33.jpg						
10	https://0ae1006704b845398...	GET	/image?filename=67.jpg						
12	https://0ae1006704b845398...	GET	/image?filename=58.jpg						
16	https://0ae1006704b845398...	GET	/academyLabHeader						
17	https://0ae1006704b845398...	GET	/image?filename=6.jpg						
18	https://0ae1006704b845398...	GET	/image?filename=50.jpg						
19	https://0ae1006704b845398...	GET	/image?filename=66.jpg						
20	https://0ae1006704b845398...	GET	/image?filename=15.jpg						
21	https://0ae1006704b845398...	GET	/image?filename=59.jpg						

Request

Pretty Raw Hex

```

1 GET /image?filename=10.jpg HTTP/2
2 Host: 0ae1006704b8453983109c7d007400f1.web-security-
3 Cookie: session=0UXsrW48fIIGC1btIYigVYIA0I2ldUcd
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64;
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.
  Safari/537.36
8 Sec-Ch-Ua-Mobile: ?0
9 Accept:
  image/avif,image/webp,image/apng,image/svg+xml,image
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: no-cors

```

Event log All issues

https://0ae1006704b845398310...emy.net/image?filename=10.jpg

Add to scope

Scan

Send to Intruder Ctrl+I

Send to Repeater Ctrl+R

Send to Sequencer

Send to Organizer Ctrl+O

Send to Comparer (request)

Send to Comparer (response)

Open response in browser

Request in browser >

Record an issue [Pro version only] >

Engagement tools [Pro version only] >

Show new history window

Add notes

Highlight >

Delete item

Clear history

Copy URL

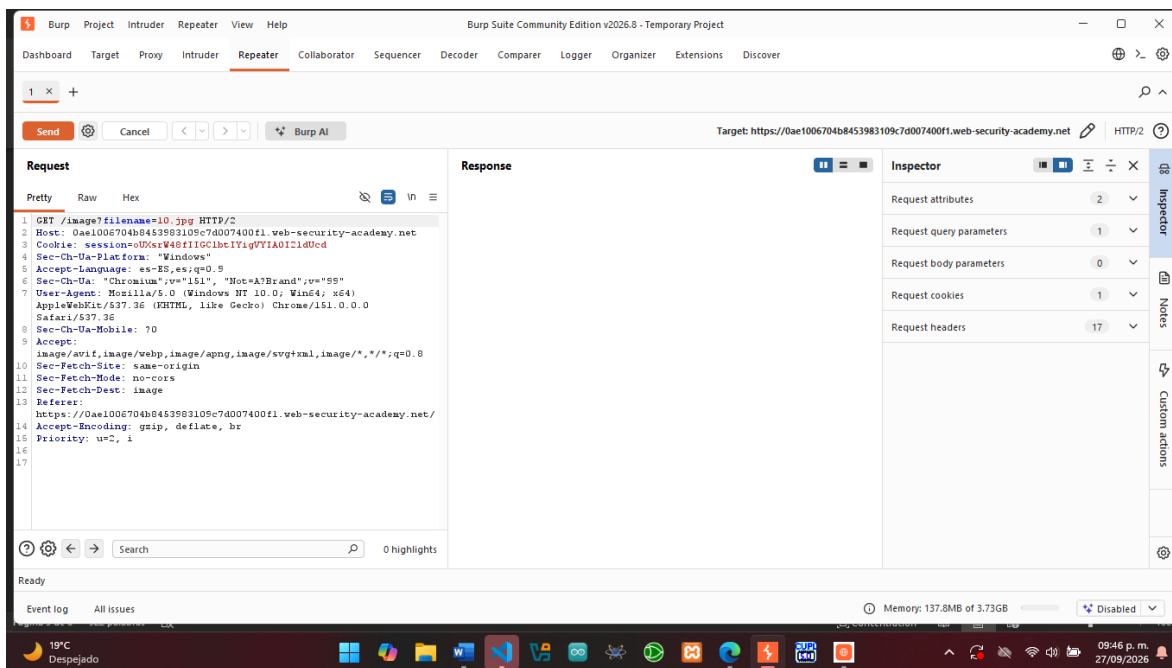
Copy as curl command (bash)

Copy links in response

Save item

Proxy history documentation

De esta manera, Burp Suite copió la petición al módulo **Repeater**, permitiendo modificar manualmente sus parámetros y reenviarla al servidor para analizar las respuestas obtenidas.



4. Modificación del parámetro vulnerable

Dentro de **Repeater**, se modificó el valor original del parámetro **filename**, sustituyendo el nombre de la imagen por el siguiente payload: **../../../../etc/passwd%00.png**.

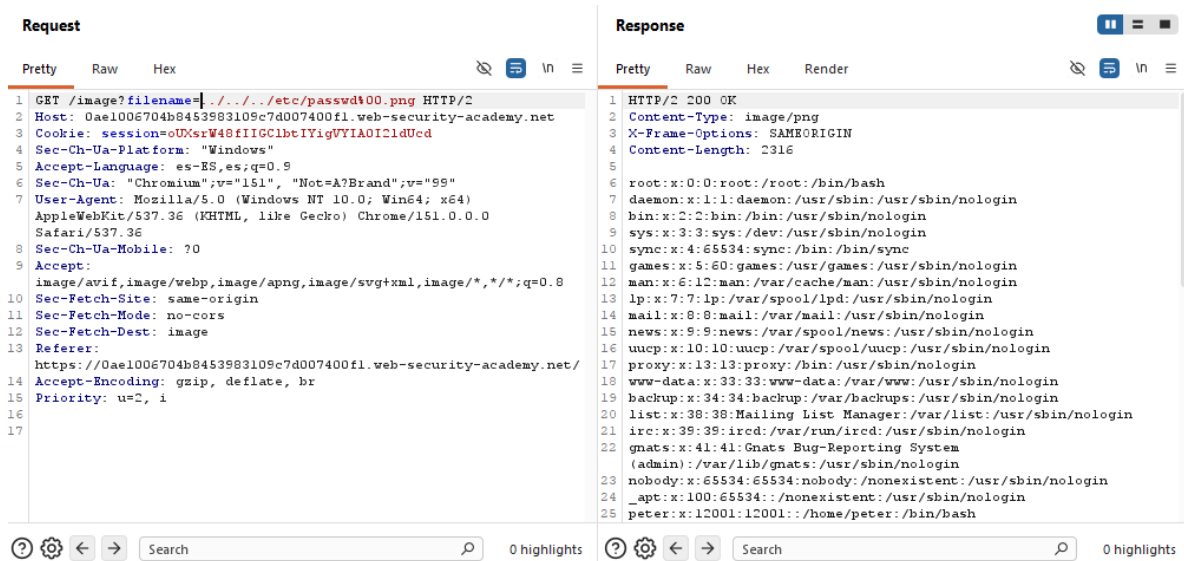
Posteriormente se envió la petición modificada al servidor.

La petición modificada quedó conceptualmente de la siguiente forma: **GET /image?filename=../../../../etc/passwd%00.png**.

5. Análisis de la respuesta

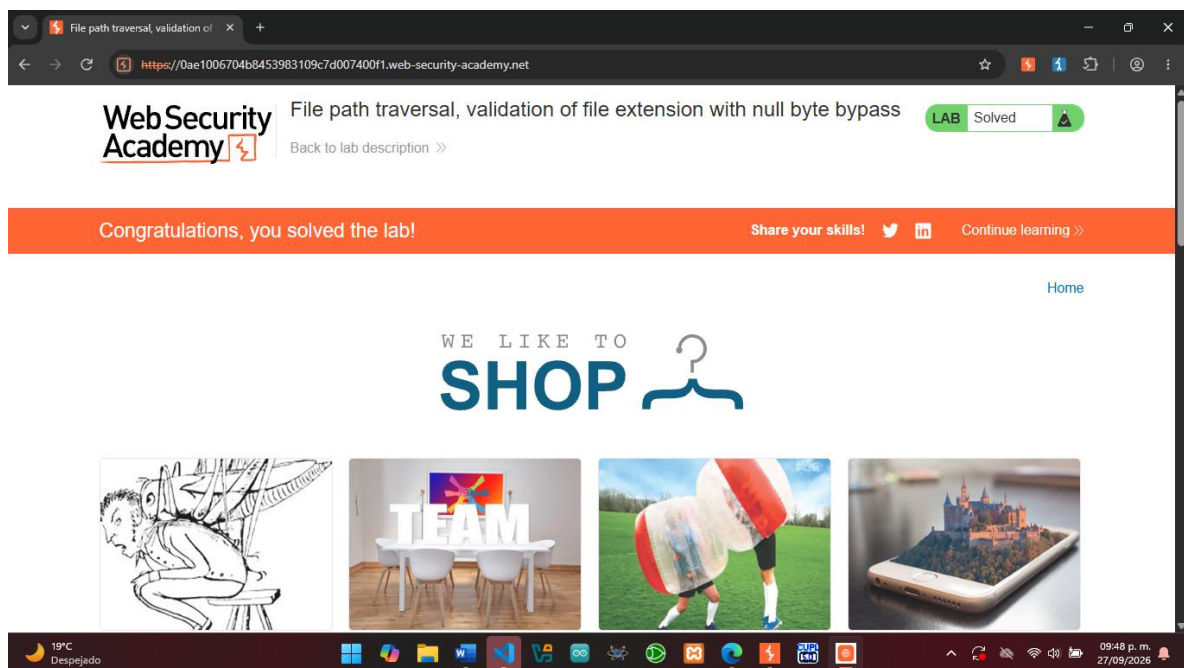
El servidor respondió con: **HTTP/2 200 OK**.

En el cuerpo de la respuesta se observó información correspondiente al archivo **/etc/passwd**.



6. Confirmación de resolución

Finalmente, se regresó a la instancia del laboratorio en PortSwigger Web Security Academy. La plataforma mostró el estado **Solved** junto con el mensaje de confirmación correspondiente, verificando que el objetivo del laboratorio había sido completado satisfactoriamente.



Explicación del payload

El payload utilizado para explotar la vulnerabilidad fue: `../../../../etc/passwd%00.png`

Este payload combina tres elementos principales: las secuencias de recorrido de directorios, el archivo objetivo y un **null byte** acompañado de una extensión válida.

La primera parte es: `../../../../`. Cada secuencia `../` permite retroceder un nivel dentro de la estructura de directorios. Al repetirla tres veces, se busca abandonar el directorio utilizado por la aplicación para almacenar las imágenes y alcanzar una ubicación superior dentro del sistema de archivos.

Posteriormente se incluye: `etc/passwd`. Esta parte especifica el archivo que se desea recuperar.

Sin embargo, la aplicación aplica una validación sobre la extensión del archivo. Para superar esta comprobación se incorpora: `%00.png`. `%00` representa un **null byte** codificado en la URL, mientras que `.png` proporciona la extensión que la validación espera encontrar.

La aplicación puede considerar que la entrada cumple con la validación debido a la presencia de `.png`. No obstante, en el entorno vulnerable, el **null byte** actúa como terminador durante el procesamiento posterior de la ruta, provocando que la parte situada después de `%00` no forme parte efectiva del nombre del archivo recuperado.

De esta manera, aunque la entrada contiene: `../../../../etc/passwd%00.png`, el recurso finalmente procesado corresponde a: `../../../../etc/passwd`.

El bypass funciona debido a la diferencia entre cómo se valida la cadena recibida y cómo se interpreta posteriormente la ruta, permitiendo superar la restricción de extensión y acceder a `/etc/passwd`.

Mitigación recomendada

Para prevenir este tipo de vulnerabilidad, la aplicación debe evitar utilizar directamente nombres de archivos o rutas proporcionadas por el usuario para determinar qué recursos serán recuperados del sistema.

Una medida recomendable consiste en utilizar **identificadores internos o una lista de valores permitidos (allowlist)**. En lugar de permitir que el usuario proporcione directamente una ruta, la aplicación puede asociar identificadores previamente definidos con archivos autorizados.

Cuando sea necesario procesar nombres de archivos proporcionados por el usuario, la validación no debe limitarse únicamente a comprobar que la cadena termine con una extensión determinada. También deben rechazarse caracteres nulos y otras representaciones que puedan modificar la forma en que diferentes componentes interpretan la ruta.

La aplicación debe normalizar y canonicalizar la ruta antes de validarla, y posteriormente comprobar que el recurso resultante permanezca dentro del directorio autorizado. De esta

manera, las comprobaciones de seguridad se realizan sobre la misma ruta que finalmente será utilizada para acceder al sistema de archivos.

Cuando la aplicación únicamente deba servir imágenes, también debe verificarse el tipo de archivo mediante mecanismos apropiados en lugar de confiar exclusivamente en la extensión incluida en su nombre.

Finalmente, debe aplicarse el **principio de mínimo privilegio**, limitando los permisos del proceso de la aplicación únicamente a los archivos necesarios para su funcionamiento. De esta forma, incluso si una validación fuera evadida, se reduce el número de recursos que podrían ser accesibles.

Referencias

PortSwigger. (s. f.). *Path traversal*. Web Security Academy.

[Path Traversal — PortSwigger Web Security Academy](#)

PortSwigger. (s. f.). *Path traversal learning path*. Web Security Academy.

[Path Traversal Learning Path — PortSwigger](#)

López Contreras, S. (2026). *CNO IV (T37A): Seguridad Informática. Actividad 09: File Path Traversal – Traversal Sequences Stripped Non-Recursively*. Universidad Politécnica de San Luis Potosí.